

Nova: Agentic Trade-Doc Automation Strategy

Understanding Nova

What is Nova?

Nova is GoComet's intelligence layer built on top of their existing logistics operations platform. Traditional SaaS solved visibility and record-keeping - you can track a shipment, store a contract, log an invoice. But the actual decision-making of whether to approve a document or how to route an exception or when to flag a discrepancy, that still lives in people's heads. Nova brings that into the system through governed agentic workflows: AI agents that can surface information and act on it, with audit trails, business rules, and human checkpoints baked in. A dashboard tells you something is wrong. Nova decides what to do about it.

The FDE Model

Every logistics business runs on a different workflow - different approval chains, different document types, different exception rules. A horizontal AI product can automate 80% of the pattern, but the last 20% is where the real value lives, and it's different for every customer. The FDE model exists to close that gap. Unlike a traditional SaaS deployment, an FDE is an engineer embedded with the client to understand their specific workflow, translate it into structured automation, and ship production code that fits their exact process. GoComet uses this model for Nova because logistics domain logic is too varied and too messy to productize fully upfront. The FDE is the bridge between an agentic platform and a workflow that runs a client's operations.

System of Outcomes vs Record and Engagement

A system of records is the source of truth around what happened, examples would be CRM systems (e.g., Salesforce), ERP software (e.g., SAP) and HR systems (e.g., Workday). A system of Engagement is a layer that enables cross collaboration between different people over this system of record, examples would be Slack, Microsoft Teams. Both these systems still require a human to read, interpret, and act.

A System of Outcomes removes the human from the loop. It extracts what matters from the record, validates it against business rules, makes a decision, and delivers a result. Nova is a System of Outcomes because the end product isn't data or a notification, it's a completed action or a flagged exception that already has a recommended resolution attached.

Problem Statement

Failure Modes in Current Flow

The trade-doc validation pipeline has five stages: ingestion, classification, extraction, rule matching, and validation with exception resolution. Traditional SaaS has partially solved the first two - collating documents and classifying them. Extraction is partially solved, the last two are still human intelligence layers where the process breaks.

- Extraction is fragile - NER and template-based parsers break on layout changes, formatting variations, and synonym fields.
- Rule matching is brittle - businesses write rigid string-matching rules that break on formatting variations, abbreviations, or field reordering. More critically, rules that depend on domain knowledge - incoterms implications, country-specific compliance requirements, ever-changing customs regulations - live entirely in people's heads and can't be encoded in a static ruleset.

- Validation suffers the same problem. A human reads every field against what they remember the customer requires. Exception resolution is worse - someone has to recall case history, interpret the discrepancy, decide the right fix, and type out the amendment manually. There's no audit trail, no structured record of why a decision was made. And CG bandwidth becomes a hard ceiling - shipment throughput can't scale without hiring more people.

Success for a CG Operator

Success for a CG operator in their first 5 minutes: they open one screen and see every shipment that arrived overnight - auto-approved ones already cleared, flagged ones showing exactly which field failed, why, and a draft amendment ready to review and send. No email triage, no manual field-checking, no rule recall. Their job becomes exception judgment, not exception discovery.

Users & Jobs-to-be-Done

Personas

- **Persona 1: Cargo Operator (CG):** Responsible for validating every inbound document against customer-specific requirements before a shipment can be cleared. Their day starts with an inbox of PDFs and ends when every case is either cleared or has an amendment request sent back. They care about one thing: knowing exactly what's wrong with a case the moment they open it, without having to read every field themselves.
- **Persona 2: Supply Chain Manager (SU):** Responsible for generating and submitting shipment documents on time. Their world is physical - loading schedules and transit windows. Paperwork is overhead. They submit documents and consider their job done. When an amendment request arrives 12 hours later asking them to fix a field mid-transit, it's a disruption they didn't budget for. They care about submitting right the first time and getting confirmation fast.

JTBD Statements

1. **SU | When** I submit my shipment documents, **I want to** be notified immediately if any field fails the customer's requirements, **so that** I can fix it before it delays the shipment rather than getting an amendment request 12 hours later.
2. **CG | When** I start my workday, **I want to** see every incoming shipment pre-sorted into approved, flagged, and needs-review - with reasoning already attached, **so that** I spend my time on judgment calls, not on reading PDFs field by field.
3. **CG | When** I need to send an amendment request back to a supplier, **I want** a pre-drafted email listing every discrepancy ready for my review, **so that** I can send it in one click instead of typing it out manually.
4. **CG | When** a new customs regulation or customer rule comes into effect, **I want** the system to flag it and revalidate affected shipments automatically, **so that** I'm not approving docs against outdated rules without realising it.
5. **SU | When** I receive an amendment request from CG, **I want** to see the full history of this shipment - what was submitted, what failed, and exactly what needs to change, **so that** I can make the corrections in one pass and resubmit without going back and forth.

Agent Architecture

Why 3 Agents?

Why not 1 Prompt - One prompt fails for three reasons: context bleed between tasks introduces hallucinations, token cost scales badly, and a single failure means rerunning everything. Three agents gives you independent failure domains - if extraction fails on a bad scan, you retry just that node without touching validation or routing.

Why 3 Agents - Each agent has a different characteristic. The Extractor needs low temperature - it's grounding facts from a document, not reasoning. The Validator needs rule application with some semantic flexibility - matching "Consignor" to "Shipper" requires judgment. The Router needs reasoning, weighing multiple mismatches to produce a decision. You can't tune one prompt to do all three.

Why not 5 - Five agents would over-partition. The three boundaries here are natural, extract facts, apply rules, and decide action. These map cleanly to planner/executor/verifier framing. Splitting further, say separating confidence scoring into its own agent, adds coordination overhead and latency without a meaningful separation of responsibility.

Agent Responsibilities

Agent	Responsibility	Input	Output
Extractor	LLM-based data extraction with confidence scoring	Raw PDF/Image + field schema	JSON of extracted fields with per-field confidence scores
Validator	Rule-based & semantic validation against customer requirements	Extracted fields JSON + customer rule set + master data	Discrepancy report with match/mismatch/uncertain per field
Auditor	Decision-making & amendment drafting based on validation outcome	Discrepancy report	Decision (auto-approve / flag / amendment) + reasoning + draft email

Communication & State Management

- **Communication:** All three agents share a single Pipeline State object that travels through the LangGraph graph. No agent calls another directly, each agent reads what it needs from the state, does its work, and writes its output back. The next agent picks up from where the previous one left off.
- **State Persistence:** State is checkpointed in MongoDB after each node completes. This means if the pipeline crashes mid-run, say the Validator fails on a bad rule lookup, the system knows the last completed stage and resumes from there rather than restarting extraction from scratch. The `pipeline_status` field tracks exactly which stage the pipeline is at any point in time.

Every agent's output is auditable independently. You can inspect what the Extractor produced before the Validator ran, which is critical for debugging hallucinations or tracing a disputed decision.

LLM & Tooling Choices

Model Selection

- **Extractor Agent:** GPT-4o-vision for its superior spatial reasoning and ability to interpret complex tables.
- **Validator Agent:** Claude 3.5 Sonnet for its high speed and precise adherence to complex logistical rulesets.
- **Auditor Agent:** GPT-4o is the right call at this layer. Heaviest reasoning load: weighing multiple mismatches, producing a defensible decision, and drafting a coherent amendment email. Quality matters more than cost at the decision point.

Vision & Orchestration

- **Vision Model:** GPT-4o vision handles all document types. PDFs are rendered to images page by page before the LLM call. Fallback on bad quality: any field

returning confidence below 0.6 triggers a field-level retry with GPT-4o at higher inference settings. The full document is not rerun, only the low-confidence fields are retried. If confidence remains low after retry, the field is marked **uncertain** and surfaced to CG for manual review.

- **Orchestration:** LangGraph. Three reasons: native checkpointing between nodes means a mid-pipeline crash resumes from the last completed stage rather than restarting from scratch; conditional edges handle the Router's branching logic cleanly, auto-approve, flag, and amendment each map to a different downstream path; and model switching across providers requires no orchestration rewrite. A custom framework would require rebuilding all three of these from scratch.
- **Structured Output:** Structured output is used in the Extractor, the field schema is enforced via a Pydantic model so every output is guaranteed to have value and confidence per field. It is also used in the Validator to enforce the discrepancy report schema. Structured output is explicitly avoided in the Router's amendment draft - that output is free-form natural language and constraining it to a schema would produce robotic, unusable emails.
- **Tool Use:** Tool use is used in the Validator for semantic knowledge base lookups - country-specific compliance rules and handling multiple customer's specific requirements will be too large and too dynamic to inject into the prompt directly, so the Validator calls a retrieval tool to fetch only the relevant rules for each field being checked.

Trust, Failure Handling & Evals

- **Hallucination prevention:** The Extractor is required to return a source snippet alongside every extracted value, the exact text from the document where the field was found. If the model cannot ground a value in the document, it cannot return high confidence. Hallucinated fields naturally surface as low confidence and get flagged for human review rather than passing through silently.

- **Low confidence handling:** Any field with confidence below 0.6 triggers a field-level retry with a stronger model. Only that field is retried, not the full document. If confidence remains below threshold after retry, the field is marked **uncertain** and surfaced explicitly to CG. There is no silent approval path in the pipeline.
- **Agent loops and runaway costs:** Two hard limits at the graph level, a max iteration count per node, and a max token budget per pipeline run. If either limit is hit, the pipeline fails loudly with a clear error state written to **pipeline_status**. The run is flagged for human review rather than continuing blindly. Langfuse traces every token spent per run, so cost anomalies are visible before they compound.
- **Evals:**
 - **Offline eval:** A labelled test set of 20-30 trade documents where ground truth field values are known. The Extractor is run against this set and scored on field-level accuracy and confidence calibration, do high confidence fields actually have higher accuracy than low confidence ones? This runs on every model or prompt change before deployment.
 - **Online metric:** Amendment request rate, the percentage of shipments that required a human to send a correction back to SU after the agent processed them. Baseline today is 60-80% of shipments requiring at least one amendment cycle. Target for a 2-week pilot: below 30%.

Metrics & Success Criteria

- **North Star Metric:** Straight-through processing rate - the percentage of shipments that move from document submission to approval with zero human intervention. Target for pilot: above 50% by week 2.

- **Supporting Metrics:**

Category	Metric	Target
Agent Quality	Field-level extraction accuracy against labelled test set	Above 90%
Agent Quality	Human-in-the-loop escalation rate. % of shipments routed to manual review	Below 30%
Agent Quality	Confidence calibration. Do high confidence fields have higher accuracy	Yes, measurable correlation
System Health	Pipeline failure rate. % of runs that errored before completion	Below 5%
System Health	Cost per document. Average token spend per pipeline run	Below \$0.05
Business Outcome	Amendment cycle reduction. Avg number of SU-CG correction loops per shipment	Down from 2-4 to below 1
Business Outcome	CG daily throughput. Documents processed per operator per day	2x baseline

- **Go/No-Go criteria for 2-week pilot:**

- Go if:
 - STP rate is above 50%
 - Amendment cycle rate is below 1 per shipment on average.
 - Zero cases of silent approval. Every uncertain field surfaced.
- No-Go if:
 - STP rate is below 25% after 2 weeks.

- CG operators are manually overriding agent decisions more than 50% of the time.
- Any hallucinated field value passed through without being flagged.

What's Next

This POC was built for speed, several choices would change in production.

1. Storage moves to **Postgres** for reliability and multi-tenant isolation.
2. The customer rule retrieval moves from a hardcoded dict to a **proper RAG pipeline** over a continuously updated **knowledge base**, pulling only relevant rules per field rather than loading everything into context.
3. **Extraction gets a hybrid layer**: if a vendor layout has been seen before and classified, a lightweight OCR/NER solution handles it cheaply. Unknown layouts fall through to the vision LLM. This alone would cut extraction cost significantly at scale.
4. **Cross-document validation** is the most important missing piece. Right now each document is validated independently. A real shipment arrives as a set - Bill of Lading, Commercial Invoice, Packing List - and fields like consignee and HS code must be consistent across all three. That consistency check needs to be added as a fourth agent or an additional Validator pass.
5. Finally, **a feedback loop**. When CG overrides an agent decision or edits an amendment draft, that correction should flow back into the system - improving rule sets, flagging prompt weaknesses, and building a labelled dataset for **offline evals over time**. Without this, the system is static. With it, it gets better every week.